

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (6)
 - aula_mat
 - bdaula
 - biblioteca
 - clima_alerta
 - postgres
 - sistema_bancario
 - Login/Group Roles
 - Tablespaces

SQL X Statistics X Dependencies X Dependents X Processes X bdaula/postgres@... X clima_alerta/postg... X biblioteca/postgres@PostgreSQL 18* X

Query History

```
73 returns trigger language plpgsql as $$ begin
74
75 if new.quantidade > 100 then raise exception 'A quantidade não pode ser maior que 100';
76 end if;
77
78 return new;
79 end; $$;
80
81 create trigger trg_valid_limit
82 before update on livro
83 for each row
84 execute function validade_limit_estq()
85
86 update livro
87 set quantidade = 20
88 where id_livro = 1;
89
90
91 update livro
92 set quantidade = 120
93 where id_livro = 1;
94
95
```

Data Output Messages Notifications

ERROR: A quantidade não pode ser maior que 100
CONTEXT: PL/pgSQL function validade_limit_estq() line 3 at RAISE

ERRO: A quantidade não pode ser maior que 100
SQL state: P0001

Total rows: Query complete 00:00:00.063 CRLF Ln 89, Col 1

20°C Pred. nublado

Pesquisar

POR PTB 100% 12:59 11/06/2026

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (6)
 - aula_mat
 - bdaula
 - biblioteca
 - clima_alerta
 - postgres
 - sistema_bancario
 - Login/Group Roles
 - Tablespaces

SQL X Statistics X Dependencies X Dependents X Processes X bdaula/postgres@... X clima_alerta/postg... X biblioteca/postgres@PostgreSQL 18* X

Query History

```
70 -- EXERCICIO 3 --
71
72 create or replace function validade_limit_estq()
73 returns trigger language plpgsql as $$ begin
74
75 if new.quantidade > 100 then raise exception 'A quantidade não pode ser maior que 100';
76 end if;
77
78 return new;
79 end; $$;
80
81 create trigger trg_valid_limit
82 before update on livro
83 for each row
84 execute function validade_limit_estq()
85
86 update livro
87 set quantidade = 20
88 where id_livro = 1;
89
90
91 update livro
92 set quantidade = 120
93 where id_livro = 1;
94
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 62 msec.

Total rows: Query complete 00:00:00.062 CRLF Ln 92, Col 21

20°C Pred. nublado

Pesquisar

POR PTB 100% 12:59 11/06/2026

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (6)
 - aula_mat
 - bdaula
 - biblioteca
 - clima_alerta
 - postgres
 - sistema_bancario
 - Login/Group Roles
 - Tablespaces

SQL X Statistics X Dependencies X Dependents X Processes X bdaula/postgres@... X clima_alerta/postg... X biblioteca/postgres@PostgreSQL 18*

Query History

```
-- EXERCICIO 3 --
71
72 create or replace function validade_limit_estq()
73 returns trigger language plpgsql as $$ begin
74
75 if new.quantidade > 100 then raise exception 'A quantidade não pode ser maior que 100';
76 end if;
77
78 return new;
79 end; $$;
80
81 create trigger trg_valid_limit
82 before update on livro
83 for each row
84 execute function validade_limit_estq();
85
86 update livro
87 set quantidade = 20
88 where id_livro = 1;
89
90 update livro
91 set quantidade = 80
92 where id_livro = 1;
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 89 msec.

Total rows: Query complete 00:00:00.089 CRLF Ln 86, Col 1

20°C Pred. nublado

Pesquisar

POR PTB 100% 12:59 11/06/2026

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (6)
 - aula_mat
 - bdaula
 - biblioteca
 - clima_alerta
 - postgres
 - sistema_bancario
 - Login/Group Roles
 - Tablespaces

SQL X Statistics X Dependencies X Dependents X Processes X bdaula/postgres@... X clima_alerta/postg... X biblioteca/postgres@PostgreSQL 18*

Query History

```
46 create table log_livro(id_log serial primary key,
47 titulo varchar (255),
48 data_exclusao timestamp,
49 mensagem varchar(255)
50 );
51
52 create or replace function log_excl_livro()
53 returns trigger language plpgsql as $$ begin
54
55 insert into log_livro(titulo, data_exclusao,mensagem)
56 values (old.titulo, now(), 'Livro removido do sistema!');
57
58 return old;
59 end; $$;
60
61 create trigger trg_log_excl
62 after delete on livro
63 for each row
64 execute function log_excl_livro();
65 delete from livro where id_livro = 15;
66
67 select * from livro;
```

Data Output Messages Notifications

Showing rows: 1 to 9 Page No: 1 of 1

id_livro	titulo	ano_publicacao	id_autor	id_editora	num_paginas	quantidade
[PK] integer	character varying (150)	integer	integer	integer	integer	integer
1	O Hobbit	1937	1	1	150	5
2	Os Senhores dos Anéis	1954	1	1	[null]	3
3	Dom Casimiro	1889	2	2	208	0
4	A Hora da Estrela	1977	3	2	88	8
5	Dom Casimiro	1899	3	2	208	4

Total rows: 9 Query complete 00:00:00.129 CRLF Ln 67, Col 22

21°C Pred. nublado

Pesquisar

POR PTB 100% 12:51 11/06/2026

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (6)
 - aula_mat
 - bdaula
 - biblioteca
 - clima_alerta
 - postgres
 - sistema_bancario
 - Login/Group Roles
 - Tablespaces

SQL X Statistics X Dependencies X Dependents X Processes X bdaula/postgres@... X clima_alerta/postg... X biblioteca/postgres@PostgreSQL 18*

Query History

```
-- EXERCICIO 2 --
create table log_livro(id_log serial primary key,
titulo varchar (255),
data_exclusao timestamp,
mensagem varchar(255)
);

create or replace function log_excl_livro()
returns trigger language plpgsql as $$ begin
insert into log_livro(titulo, data_exclusao,mensagem)
values (old.titulo, now(), 'Livro removido do sistema!');
return old;
end; $$;

create trigger trg_log_excl
after delete on livro
for each row
execute function log_excl_livro();
delete from livro where id_livro = 15;
```

Data Output Messages Notifications

DELETE 0

Query returned successfully in 72 msec.

Total rows: Query complete 00:00:00.072 CRLF Ln 65, Col 1

21°C Pred. nublado

Pesquisar

POR PTB 100% 12:51 11/06/2026

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (6)
 - aula_mat
 - bdaula
 - biblioteca
 - clima_alerta
 - postgres
 - sistema_bancario
 - Login/Group Roles
 - Tablespaces

SQL X Statistics X Dependencies X Dependents X Processes X bdaula/postgres@... X clima_alerta/postg... X biblioteca/postgres@PostgreSQL 18*

Query History

```
-- EXERCICIO 1 --
create or replace function bloquear_exclusao()
returns trigger language plpgsql as $$ begin
if old.quantidade > 0 then
raise exception 'Não é possível excluir livro';
end if;
return old;
end; $$;

create trigger trg_bloc_exclusao
before delete on livro
for each row
execute function bloquear_exclusao();
delete from livro where id_livro = 1;
delete from livro where id_livro = 14;
```

Data Output Messages Notifications

DELETE 0

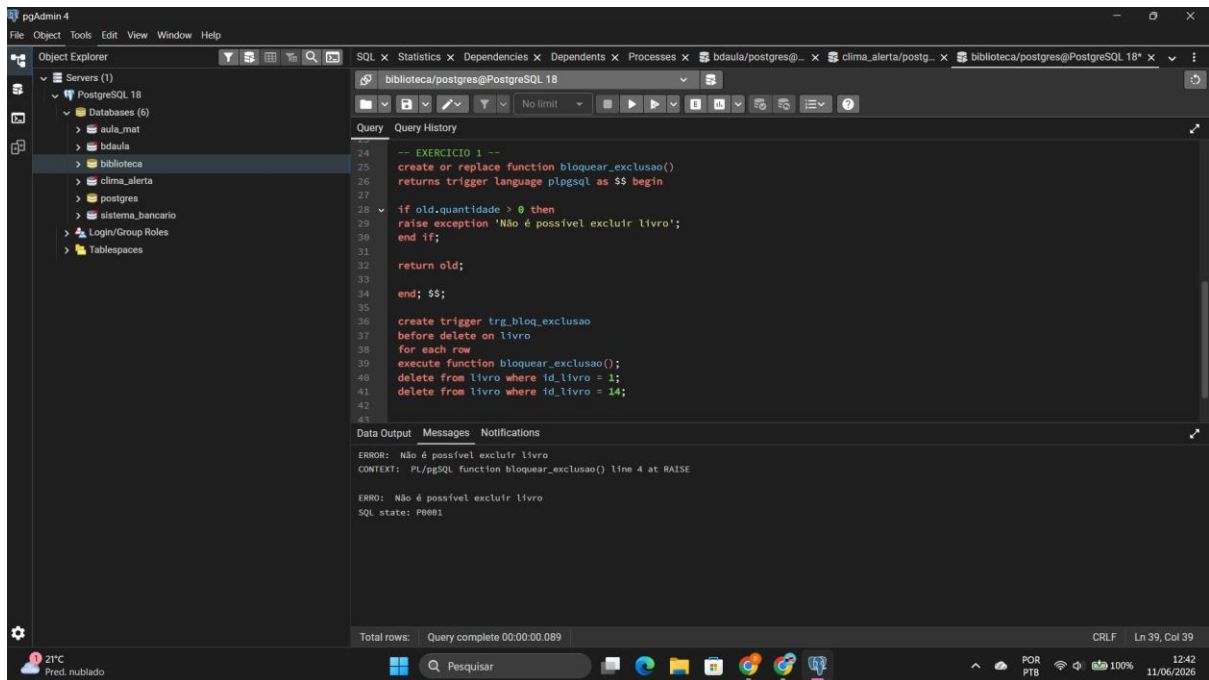
Query returned successfully in 71 msec.

Total rows: Query complete 00:00:00.071 CRLF Ln 42, Col 1

21°C Pred. nublado

Pesquisar

POR PTB 100% 12:50 11/06/2026



PERGUNTAS

a) A diferença entre triggers BEFORE e AFTER.

A principal diferença está no **momento da execução** em relação à operação principal (INSERT, UPDATE ou DELETE) e no **controle dos dados**:

- **BEFORE (Antes):** É disparada **antes** que a alteração seja gravada definitivamente na tabela. Ela permite interceptar os dados que o usuário enviou, modificá-los se necessário (usando o NEW), ou cancelar a operação inteira (usando um RAISE EXCEPTION) caso alguma regra seja violada.
- **AFTER (Depois):** É disparada **depois** que a alteração já foi realizada e validada na tabela de origem. Nesse momento, os dados já foram salvos, então a trigger não pode mais alterar os valores que foram inseridos/atualizados diretamente nela. Ela serve para reagir ao que já aconteceu.

b) Qual delas deve ser usada para validação.

Deve ser usada a trigger **BEFORE**.

Por quê? Porque validações servem para garantir a integridade dos dados (como checar se há estoque suficiente antes de vender). Se a validação falhar, a trigger BEFORE pode abortar a operação imediatamente antes que qualquer dado errado seja

salvo no banco de dados, evitando desperdício de processamento e mantendo o banco consistente.

c) Qual delas deve ser usada para auditoria.

Deve ser usada a trigger **AFTER**.

Por quê? A auditoria (ou registro de logs) serve para guardar o histórico do que **realmente aconteceu** no banco de dados. Se usássemos uma trigger BEFORE e a operação principal falhasse logo em seguida por qualquer outro motivo, o log teria registrado uma alteração que nunca existiu de verdade. Usando AFTER, você garante que só vai registrar o log se a alteração tiver sido concluída com sucesso.

d) Por que a ordem de execução é importante em bancos de dados reais.

A ordem de execução (BEFORE $\rightarrow$ Operação no Banco $\rightarrow$ AFTER) é vital em cenários reais por três motivos principais:

1. **Desempenho e Economia de Recursos:** Não faz sentido gastar processamento gravando um log de auditoria (AFTER) ou alterando outras tabelas se a validação inicial (BEFORE) falhar e cancelar tudo.
2. **Consistência dos Dados:** Garante que regras de negócio complexas (como o bloqueio de exclusão que você fez na outra questão ou a checagem de estoque) sejam processadas estritamente antes que o dado se torne visível ou definitivo para outros usuários do sistema.
3. **Segurança e Confiabilidade:** Em sistemas reais (como bancos ou e-commerces), as ações precisam ser atômicas (ou acontecem 100% certo ou não acontecem nada). A ordem correta das triggers garante que nenhuma informação inválida passe e que nenhum histórico seja registrado incorretamente.

5.

a) Quais riscos essa decisão pode gerar.

Mover todas as validações exclusivamente para a aplicação traz riscos graves à segurança e à saúde dos dados:

- **Bypass de Regras (Ignorar a validação):** Se um desenvolvedor ou administrador de banco de dados rodar um comando diretamente no terminal do banco (via DBeaver, pgAdmin, etc.) ou se houver um script de migração manual, os dados serão inseridos **sem nenhuma validação**, pois o banco aceitará qualquer coisa.

b) Quais vantagens existem em manter regras diretamente no banco.

- **Centralização da Verdade:** A regra de negócio fica em um único lugar seguro. Não importa por onde o dado tente entrar (sistema web, app, planilha importada), ele **terá** que passar pela mesma regra.
- **Segurança Absoluta (Última Linha de Defesa):** O banco de dados se torna blindado. Mesmo que o código do backend tenha um bug ou seja invadido, o banco rejeita dados corrompidos.
- **Performance em Operações em Lote (Bulk):** Se o sistema precisar atualizar 100.000 linhas de uma vez, fazer isso via aplicação exige trazer os dados, validar um por um e mandar de volta. A trigger faz isso direto no motor do banco de forma infinitamente mais rápida.
- **Múltiplas Aplicações Inconsistentes:** Em ambientes corporativos, é comum ter vários sistemas acessando o mesmo banco (o app mobile, o sistema web, um sistema de BI). Se a validação estiver só na aplicação web, o app mobile pode inserir dados inválidos por falha de replicação de código.
- **Inconsistência por Concorrência:** Duas requisições da aplicação podem tentar vender o último item do estoque ao mesmo tempo. Sem uma trava direta no banco (como uma trigger ou constraint), a aplicação pode validar que "há estoque" para as duas, gerando uma venda duplicada indevida.

c) Explique como triggers ajudam na integridade e consistência dos dados.

As triggers agem como "vigilantes automatizados" e garantem a consistência por meio de três pilares:

1. **Automação Incondicional:** Elas não dependem de um comando humano ou de um "lembrete" no código do backend. Elas são disparadas por eventos do próprio ciclo de vida do dado (INSERT, UPDATE, DELETE).
2. **Garantia de Estado Válido:** Com o uso de BEFORE, o banco garante que um dado só mude de estado se satisfizer os critérios pré-estabelecidos (ex: o saldo de uma conta nunca ficar menor que o limite permitido).
3. **Cascatas Complexas:** Elas conseguem manter tabelas relacionadas atualizadas em tempo real de forma atômica (tudo acontece na mesma transação). Se a trigger falhar no meio do caminho, o banco dá *rollback* e nada é corrompido.

d) Cite um exemplo real de uso de triggers em sistemas corporativos.

Um exemplo clássico e vital é o **Controle de Estoque e Reserva de Produtos em um E-commerce:**

O cenário: Quando um cliente finaliza a compra de um item, a aplicação dispara um INSERT na tabela `itens_pedido`.

A Trigger em ação: Uma trigger BEFORE INSERT entra em ação, verifica se a quantidade solicitada existe na tabela `produtos_estoque`. Se houver estoque, ela automaticamente dá um UPDATE subtraindo os itens do estoque e permite a compra. Se o estoque for insuficiente, ela dispara um RAISE EXCEPTION, cancelando a compra e impedindo que o sistema venda um produto que não existe.